Is PPO inherently safe?

Estelle Ngounou (300269700)
Eliel Beonao (300326742)
Jean-Philippe Nahimana Bahenda (300324295)

CSI 4900

SUPERVISOR:
Colin Bellinger

April 30th 2026
Winter 2026
Engineering Faculty
University of Ottawa

1. Abstract

This paper investigates whether safe behavior emerges naturally in agents trained with Proximal Policy Optimization (PPO), or whether explicit constraint enforcement is necessary. We train a Brax Ant robot on a 2D navigation task under three constraint conditions: no constraint, soft constraint, and hard constraint. Each model is trained for 100M timesteps across 5 independent seeds under identical environment conditions. The obtained results show that PPO does not inherently learn safe behavior. The unconstrained agent accumulates unsafe behaviours despite extended training. Soft penalties marginally improve safety metrics but do not meaningfully deter unsafe behavior. Hard constraints enforce safety most effectively but at the cost of task performance. These findings suggest that safety must be explicitly designed into the reward and termination structure rather than expected to emerge from reward maximization alone.

2. Introduction & Background

Reinforcement learning is a branch of machine learning widely used for robotics applications. One of the most popular algorithms used is PPO, or Proximal Policy Optimization.

A PPO algorithm is a policy gradient method that optimizes a clipped surrogate objective to ensure stable policy updates for stable training. Agents trained with PPO can achieve very good performances across a wide range of control tasks, however performance and safety are not the same objective.

The literature on PPO mainly focuses on reward maximization and performance. Safety in its whole is overlooked, which is a problem since in safety-critical applications such as robotics, autonomous, and medical devices, an agent that maximizes reward while violating safety constraints is not deployable regardless of its task performance.

To implement the PPO algorithm, Brax was used because it is a fully differentiable rigid body physics engine implemented in JAX, designed for massively parallel simulation. The Ant morphology used in this work is a four-legged robot that must simultaneously learn locomotion and navigation, making it a non-trivial testbed for safety research.

3. Problem Formulation

The navigation task is defined in 2D. At the beginning of each episode, a goal position and three logical obstacles are sampled. The agent must navigate from its reset position to within a success radius of the goal while respecting safety constraints.

Safety constraints are defined as:

- **Obstacles collision**
- **Going out of bounds**
- **Speed violation**
- **Falling Over**

The three paradigms differ in how these constraints enter the reward and termination:

- **No constraint:** constraints are measured but do not affect reward or termination
- **Soft constraint:** constraint violations incur reward penalties but do not terminate the episode
- **Hard constraint:** constraint violations incur penalties and immediately terminate the episode

3.1 Reward Structure

The reward function is designed to encourage goal-directed behavior while discouraging inefficient or unsafe actions. It combines positive reinforcement for progress toward the goal with penalties that regulate the agent's behavior under different constraint settings.

No constraint: In the baseline configuration, the reward function focuses solely on task completion and efficiency. The agent receives a progress reward proportional to the reduction in distance to the goal at each timestep, encouraging continuous movement toward the target. A success bonus is granted upon reaching the goal, providing a strong incentive for task completion. Additionally, a small action cost is applied to penalize large control inputs, promoting smoother and more energy-efficient movements.

Soft constraint: The soft-constraint configuration builds upon the baseline reward by introducing penalties for unsafe behaviors. Specifically, the agent incurs penalties for collisions, out-of-bounds events, and falls. These penalties reduce the total reward but do not terminate the episode. As a result, the agent retains the ability to continue interacting with the environment after a violation. This creates a trade-off: the agent may still engage in unsafe actions if doing so leads to higher cumulative reward over the episode.

Hard constraint: The hard-constraint configuration uses the same reward components as the soft-constraint setting but enforces strict safety through episode termination. Any violation, whether a collision, fall, or out-of-bounds event, results in immediate termination of the episode. This mechanism introduces an implicit but strong penalty by preventing the agent from accumulating future rewards after unsafe behavior. Consequently, the agent is strongly incentivized to avoid violations entirely, as they directly limit long-term return.

3.2 Termination Conditions

Condition	No Constraint	Soft Constraint	Hard Constraint
Success	✓	✓	✓
Max steps	✓	✓	✓
Collision	✗	✗	✓
Out of bounds	✗	✗	✓
Speed violation	✗	✗	✗
Fall	✗	✗	✓

4. System Design

4.1 Environment

The environment is implemented using Brax, a differentiable physics engine designed for large-scale parallel simulation within the JAX framework. The agent is modeled as a quadrupedal Ant robot operating in a two-dimensional navigation setting.

At the beginning of each episode, the agent is initialized at a fixed starting position, while the goal location is randomly sampled from a predefined region of the environment. In addition, three obstacles are stochastically positioned between the start and goal locations.

The resulting task requires the agent to jointly learn locomotion and goal-directed navigation under spatial constraints. This combination introduces significant complexity, as the agent must simultaneously acquire stable control policies and develop obstacle-aware navigation strategies.

4.2 Observation Space

The observation space is represented by a 27-dimensional state vector that captures both the agent's proprioceptive state and relevant environmental information. Specifically, it includes joint positions and velocities, as well as global body orientation and angular velocity. Furthermore, the observation encodes the relative position of the goal and the relative positions of surrounding obstacles.

The inclusion of obstacle-related features is critical, as it enables the agent to condition its policy on environmental structure and to learn anticipatory avoidance behaviors, rather than relying solely on reactive responses following unsafe interactions.

4.3 Domain Randomization

To enhance generalization and robustness, domain randomization is applied at the episode level. In particular, goal positions are sampled from a predefined distribution, and obstacle configurations vary across episodes. Additionally, minor perturbations may be applied to the agent's initial state.

This stochastic initialization prevents overfitting to specific environment configurations and encourages the learning of policies that generalize across a diverse set of scenarios. Consequently, the agent is better equipped to handle previously unseen configurations during evaluation.

5. Methods

5.1 PPO Implementation

We implement PPO from scratch in pure JAX. The actor and critic are two fully separate MLPs. Both networks take the raw enriched observation as input independently.

Baseline Metrics:

- **Success_bonus (+40.0):** Applied when reaching the goal.
- **Step penalty (−0.03):** Applied at every timestep.
- **Collision penalty (−5.0):** Applied when the agent collides with an obstacle, discouraging unsafe navigation.
- **Out-of-bounds penalty (−5.0):** Triggered when the agent leaves the valid area.
- **Speed penalty (−0.5):** Applied for undesirable speed behavior.
- **Fall penalty (−6.0):** Given when the agent falls.

Network architecture:

- Input layer: 106-dimensional observation vector(brax ant obs, torso history, velocity, goal vec, obstacle rel xy, obstacle radii)
- Hidden layer 1: 256 units, tanh activation
- Hidden layer 2: 256 units, tanh activation
- Actor output: 8-dimensional mean vector and a learned 8-dimensional log standard deviation
- Critic output: 1 scalar state-value estimate

PPO hyperparameters:

Parameter	Value
Learning rate	3e-4
Discount factor γ	0.99

GAE λ	0.95
Clip ε	0.2
Value coefficient	0.5
Entropy coefficient	0.02
Max grad norm	1.0
PPO epochs	8
Steps per env	256
Minibatch size	4096
Max Step per Episode	500

5.2 Training Setup

- **Timesteps:** 100M per seed
- **Seeds:** 5 independent seeds (0, 1, 2, 3, 4)
- **Parallel environments:** 1024 via jax.vmap
- **Hardware:** 1 NVIDIA A100 80GB GPU per seed
- **Cluster:** Narval (Digital Research Alliance of Canada)
- **Total compute:** ~15 A100 GPU-days

5.3 Evaluation Protocol

Every 10 training iterations, the current policy is evaluated deterministically over 50 episodes. A final evaluation over 200 episodes is performed at the end of training. Metrics are averaged across seeds and reported as mean ± std.

6. Experiments and Results

6.1 Safety Violations

The no-constraint model exhibits a consistently high rate of safety violations throughout training, stabilizing at approximately 50 violations per 100 steps. This behavior indicates that, in the absence of explicit constraints, the agent does not acquire safe policies and instead prioritizes reward maximization regardless of unsafe interactions.

The soft-constraint model demonstrates behavior closely aligned with the baseline. The frequency of violations remains comparably high, suggesting that the imposed penalties are insufficient to effectively discourage unsafe actions. As a result, the agent continues to tolerate violations when they contribute to higher cumulative return.

In contrast, the hard-constraint model achieves a substantial reduction in safety violations. Since any violation results in immediate episode termination, unsafe behaviors directly limit future reward accumulation. This induces a strong implicit penalty, leading the agent to adopt significantly safer policies.

6.2 Task Performance

The no-constraint and soft-constraint models achieve comparable levels of task performance, with success rates typically ranging between 30% and 35%. This similarity indicates that the introduction of soft penalties does not significantly alter the agent's ability to reach the goal or avoid violations.

Conversely, the hard-constraint model exhibits lower success rates. This reduction can be attributed to restricted exploration, as early episode termination limits the agent's capacity to discover and refine successful trajectories. While the learned policies are safer, they are less efficient in achieving the task objective.

6.3 Learning Curves

The no-constraint model demonstrates rapid convergence; however, it converges to policies that are inherently unsafe. The absence of constraints allows the agent to quickly exploit reward signals without regard for safety.

The soft-constraint model closely mirrors the learning dynamics of the baseline, further reinforcing the observation that our penalty-based approach alone is insufficient to meaningfully alter agent behavior in this setting.

The hard-constraint model exhibits slower learning progression. The enforcement of strict termination reduces effective exploration, thereby increasing the difficulty of discovering high-performing policies.

Additionally, variance across random seeds is notably higher in the hard-constraint setting, indicating increased sensitivity to initialization and stochasticity in the training process.

7. Discussion

7.1 Safety is Not Inherent in PPO

The experimental results demonstrate that Proximal Policy Optimization (PPO) does not inherently promote safe behavior. In the absence of explicit safety constraints, the agent consistently converges to policies that maximize reward while tolerating a high rate of safety violations. This suggests that PPO, as a general-purpose policy optimization algorithm, is fundamentally agnostic to safety considerations unless they are explicitly encoded in the reward function or environment dynamics. Consequently, relying solely on standard reinforcement learning objectives is insufficient for safety-critical applications, where unsafe actions may yield high short-term rewards.

7.2 Hard Constraints Are Effective But Costly

Hard constraints emerge as the most effective mechanism for enforcing safety, as they directly prevent unsafe behaviors through immediate episode termination. This introduces a strong implicit penalty by eliminating the possibility of future reward accumulation following a violation. As a result, the agent is compelled to learn policies that strictly avoid unsafe states.

However, this effectiveness comes at a significant cost. Early termination reduces the effective exploration space, limiting the agent's ability to discover and refine successful trajectories. This leads to slower convergence and lower overall task performance. Furthermore, the increased variance across training runs suggests that hard constraints amplify sensitivity to stochastic factors, making training less stable.

7.3 The Safety-Performance Tradeoff

The results highlight a clear trade-off between safety and performance. While the no-constraint and soft-constraint models achieve higher success rates, they do so at the expense of safety, exhibiting frequent violations. In contrast, the hard-constraint model significantly reduces unsafe behavior but suffers from decreased task performance due to restricted exploration.

This trade-off underscores a fundamental challenge in reinforcement learning: optimizing for safety and performance simultaneously is non-trivial. Methods that strongly enforce safety may hinder learning efficiency, whereas methods that prioritize performance may fail to prevent unsafe behavior. Balancing these objectives remains an open problem and is critical for real-world deployment.

7.4 Reward Scale Matters

The findings also emphasize the importance of reward scaling in shaping agent behavior. In the soft-constraint setting, the penalty terms were insufficient to meaningfully influence the policy, as the agent continued to prioritize progress toward the goal despite incurring violations. This indicates that the relative magnitude of penalties compared to positive rewards plays a crucial role in determining whether safety constraints are effectively enforced.

If penalties are too small, they are effectively ignored; if they are too large, they may dominate the learning signal and hinder task performance as it causes the agent to just freeze. Therefore, careful tuning of reward components is essential to achieve a desirable balance between safety and efficiency.

8. Conclusion

This study investigated the extent to which safety emerges in reinforcement learning when using Proximal Policy Optimization (PPO) under different constraint formulations. The results demonstrate that safety is not an inherent property of the learning process. In the absence of constraints, the agent consistently converges to unsafe policies that prioritize reward maximization.

Introducing soft constraints through penalty-based rewards does not significantly alter this behavior, as the agent continues to tolerate violations when they contribute to higher cumulative return. In contrast, hard constraints enforced through episode termination prove effective at reducing unsafe behavior by imposing a strong explicit penalty on violations.

However, this improvement in safety comes at the cost of reduced task performance and slower learning, highlighting a fundamental trade-off between safety and efficiency. Overall, these findings emphasize the need for more principled approaches to safety in reinforcement learning, particularly for applications where reliability and robustness are critical.

9. Future Work

Future research can build upon this work by exploring more advanced approaches to balancing safety and performance in reinforcement learning.

One promising direction is the use of smaller environment steps combined with stronger or adaptive penalty scaling mechanisms (Lagrangian method), where the magnitude of penalties evolves during training to better influence agent behavior. Additionally, constrained reinforcement learning frameworks, such as Constrained Policy Optimization (CPO), could provide a more principled way to enforce safety constraints while preserving performance, as it directly optimizes within a feasible region defined by safety thresholds rather than relying on penalty terms alone.

Another avenue involves the integration of risk-sensitive objectives, which explicitly account for uncertainty and worst-case outcomes rather than optimizing expected return alone. Improvements in observation design may also enhance the agent's ability to anticipate and avoid obstacles, leading to more robust policies.

Finally, extending this work to more complex environments or real-world robotic systems would provide valuable insights into the scalability and practical applicability of these approaches in safety-critical domains.

10. References

- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. *arXiv:1707.06347*

- Freeman, C. D., Frey, E., Raichuk, A., Girgin, S., Mordatch, I., & Bachem, O. (2021). Brax – A Differentiable Physics Engine for Large Scale Rigid Body Simulation. *NeurIPS 2021*
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press. Retrieved from http://incompleteideas.net/book/the-book-2nd.html
- Szepesvári, C. (2010). *Algorithms for Reinforcement Learning*. Morgan & Claypool. Retrieved from https://sites.ualberta.ca/~szepesva/papers/RLAlgsInMDPs.pdf
- OpenAI. (n.d.). *Spinning Up in Deep Reinforcement Learning*. Retrieved from https://spinningup.openai.com/en/latest/
- Levine, S. (n.d.). *CS285: Deep Reinforcement Learning*. University of California, Berkeley. Retrieved from https://rail.eecs.berkeley.edu/deeprlcourse/
- Silver, D. (n.d.). *Reinforcement Learning Course* [Video lectures]. Retrieved from https://www.youtube.com/playlist?list=PLqYmG7hTraZDM-OYHWgPebj2MfCFzFObQ
- Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). *Stable-Baselines3: Reliable Reinforcement Learning Implementations*. Retrieved from https://stable-baselines3.readthedocs.io/en/master/
- PyTorch Contributors. (n.d.). *TorchRL Documentation*. Retrieved from https://docs.pytorch.org/rl/stable/index.html
- CleanRL Contributors. (n.d.). *CleanRL Documentation*. Retrieved from https://docs.cleanrl.dev
- Gymnasium Contributors. (n.d.). *Gymnasium: Standard API for Reinforcement Learning Environments*. Retrieved from https://gymnasium.farama.org/environments/mujoco/
- Todorov, E., Erez, T., & Tassa, Y. (2012). *MuJoCo: A physics engine for model-based control*. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS).
- DeepMind. (n.d.). *MuJoCo Menagerie*. Retrieved from https://github.com/google-deepmind/mujoco_menagerie
- Kober, J., Bagnell, J. A., & Peters, J. (2013). *Reinforcement Learning in Robotics: A Survey*. *The International Journal of Robotics Research, 32*(11), 1238–1274.
- Arulkumaran, K., Deisenroth, M. P., Brundage, M., & Bharath, A. A. (2017). *Deep Reinforcement Learning: A Brief Survey*. *IEEE Signal Processing Magazine, 34*(6), 26–38.
- Li, Y. (2021). *Deep Reinforcement Learning in Robotics: A Survey*. arXiv preprint arXiv:2108.02039.
- Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., & Abbeel, P. (2017). *Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World*. IROS.
- Polydoros, A. S., & Nalpantidis, L. (2017). *Survey of Model-Based Reinforcement Learning: Applications on Robotics*. *Journal of Intelligent & Robotic Systems*.
- Levine, S., Finn, C., Darrell, T., & Abbeel, P. (2016). *End-to-End Training of Deep Visuomotor Policies*. *Journal of Machine Learning Research*.
- Kalashnikov, D., et al. (2018). *QT-Opt: Scalable Deep Reinforcement Learning for Vision-Based Robotic Manipulation*. arXiv preprint arXiv:1806.10293.